

CM05 - Libraries and Control Structures

I/O library functions, formats, conditions, switch, loops

Louis Ledoux

2026-07-05

Table of contents

1 Libraries and Basic I/O	1
1.1 Main Path	2
1.2 Worked Example	2
1.3 Anecdote Or Motivation	3
1.4 Check Question	3
1.5 Backup Index	3
1.6 Backup: Basic components of C language: Libraries	6
1.7 Backup: Libraries	7
1.8 Backup: Two basic I/O functions!	8
1.9 Backup: Formated write: printf(...)	9
1.10 Backup: Formated read: scanf(...)	9
1.11 Backup: Format of I/O expressions	10
1.12 Backup: Example	11
1.13 Backup: What happens if we use a wrong format?	11
1.14 Backup: Type conversion: Inbuilt typecast functions	12
1.15 Backup: Let's...	13
2 Control Structures	13
2.1 Main Path	13
2.2 Worked Example	14
2.3 Anecdote Or Motivation	14
2.4 Check Question	15
2.5 Backup Index	15
2.6 Backup: Basic components of C language: Control structures	18
2.7 Backup: Control structures	18
2.8 Backup: Conditions: If	19
2.9 Backup: Conditions: ternary instruction	20
2.10 Backup: Conditions: switch - case	21
2.11 Backup: Loops	21
2.12 Backup: Loops: Control statements	22
2.13 Backup: Other types of objects	23

1 Libraries and Basic I/O

Live pacing: about 8 min

Question. How does a C program communicate and choose a path?

Core claim. Library calls and control structures turn values into observable behavior.

Target capability. Students can select the right format specifier, branch, and loop for a small task.

1.1 Main Path

- Start from the question: How does a C program communicate and choose a path?
- Build the model: Library calls and control structures turn values into observable behavior.
- End with a checkable action: Students can select the right format specifier, branch, and loop for a small task.
- Keep only this slide on the horizontal path during the live lecture.

- Basic components of C language: Libraries

#181

1.2 Worked Example

- Use this as the live trace: Read one value, validate it, print a formatted result, then add a loop for repeated input.
- Representative source slide: 183
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Two basic I/O functions!

```
We have two main functions for read and write to the standard input and output
Write: int printf(formate 1,e2,...,en);
Read: int scanf(formata1,a2,...,an );
To use them we need to add the directive
#include <stdio.h>
```

#183

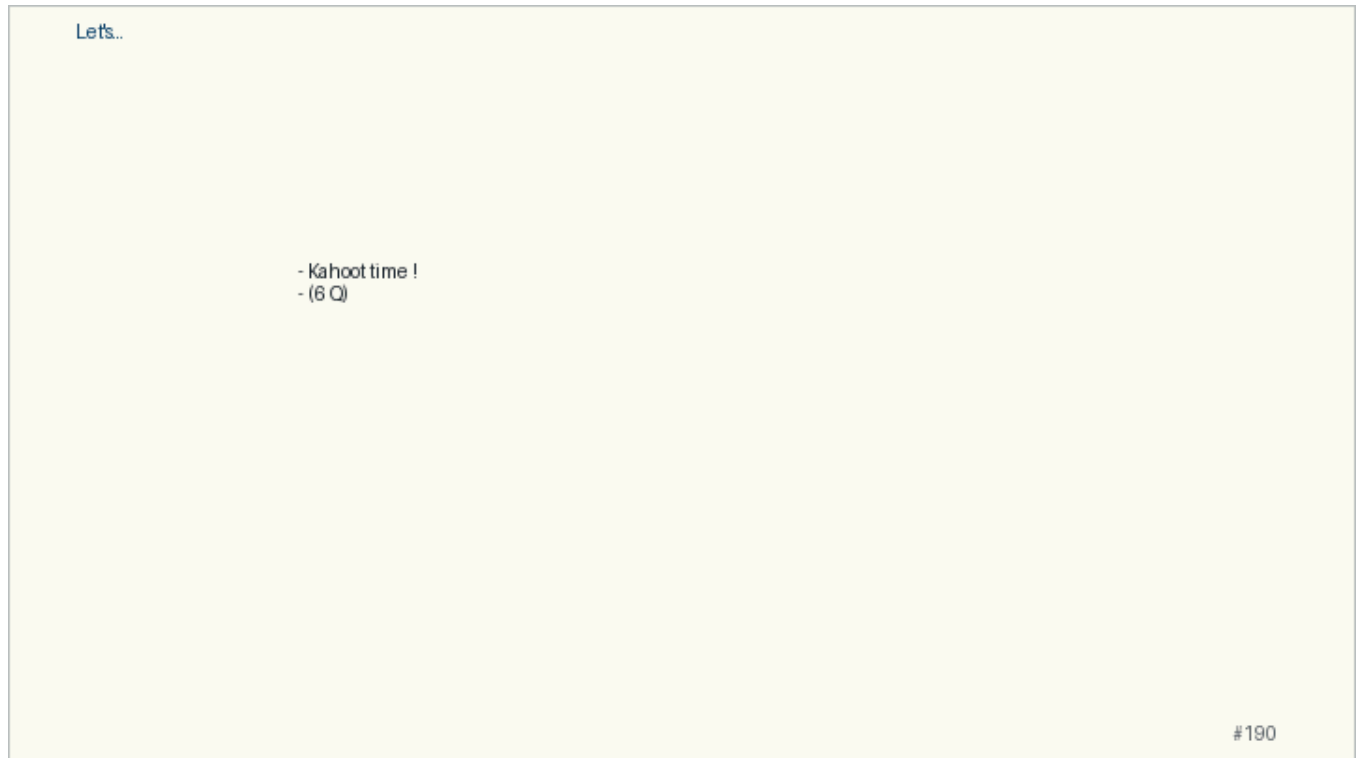
1.3 Anecdote Or Motivation

`printf` is a good warning: the compiler trusts the format string more than the value.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



1.5 Backup Index

Original slides 181-190

- Basic components of C language: Libraries

#181

slide 181

Libraries

In C, there are a lot of functions already implemented into libraries
Include system calls
Mathematical computations
...
The programmer can and should use them !
Add corresponding directive to the head of the compilation file
`#include <stdio.h>`
`#include <stdlib.h>`
`#include <math.h>`
`#include <string.h>`
...

#182

slide 182

Two basic I/O functions!

We have two main functions for read and write to the standard input and output
Write: `int printf(formate1,e2,...,en);`
Read: `int scanf(formata1,a2,...,an );`
To use them we need to add the directive
`#include <stdio.h>`

#183

slide 183

Formatted write: `printf(...)`

Syntax

`int printf(formate1,e2,...,en);`

Description

Writes (displays) the parameters `e1,e2,...,en` to the standard output based on the parameter format

The format consists of I/O expressions and, potentially, text

Each I/O expression describes how the writing of one parameter `ei` will be done using the character %

It can have a variable number of parameters, but

The number of `ei` has to correspond to the number of I/O expressions specified in the format

Result

Number of displayed characters (value < 0 in case of an error)

#184

slide 184

Formatted read: scanf(...)

Syntax

```
int scanf(format a1,a2,...,an);
```

Description

Reads from the standard input (keyboard) the values and interprets them based on the parameter

format

It assigns each read value to the variables, whose addresses are given by the parameters

a1,a2,...,an

Address of a variable is noted by the operand &nom_variable

It can have a variable number of parameters, but

The number of ai has to correspond to the number of I/O expressions specified in the format

Result

Returns the number of elements correctly assigned

#185

slide 185

Format of I/O expressions

%d: implies that the I/O is a signed integer (int)

%c: implies that the I/O is an ASCII character (char)

%f: implies that the I/O is a float (float).

%lf: implies that the I/O is a double (double).

%x: implies that the I/O is a unsigned integer to be printed using the hexadecimal representation

%s: implies that the I/O is a string

....

More info: man format or google printf

Remarks

The format can also have normal text (whatever does not have the % character)

#186

slide 186

Anchors: Basic components of C language: Libraries; Libraries; Two basic I/O functions!; Formated write: printf(...); Formated read: scanf(...); Format of I/O expressions

1.6 Backup: Basic components of C language: Libraries

Original slide 181

- Basic components of C language: Libraries

#181

1.7 Backup: Libraries

Original slide 182

In C, there are a lot of functions already implemented into libraries

Include system calls

Mathematical computations

...

The programmer can and should use them !

Add corresponding directive to the head of the compilation file

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <string.h>
```

...

Libraries

In C, there are a lot of functions already implemented into libraries
Include system calls
Mathematical computations
...
The programmer can and should use them !
Add corresponding directive to the head of the compilation file
`#include <stdio.h>`
`#include <stdlib.h>`
`#include <math.h>`
`#include <string.h>`
...

#182

1.8 Backup: Two basic I/O functions!

Original slide 183

We have two main functions **for** read and write to the standard input and output:

Write: `int printf(format, e1, e2, ..., en);`

Read: `int scanf(format, a1, a2, ..., an );`

To use them we need to add the directive

`#include <stdio.h>`

Two basic I/O functions!

We have two main functions for read and write to the standard input and output
Write: `int printf(format, e1, e2, ..., en);`
Read: `int scanf(format, a1, a2, ..., an );`
To use them we need to add the directive
`#include <stdio.h>`

#183

1.9 Backup: Formated write: printf(...)

Original slide 184

Syntax

```
int printf(format,e1,e2,...,en);
```

Description

Writes (displays) the parameters $e_1, e_2, \dots, e_n$ to the standard output based on the parameter format

The format consists of I/O expressions and, potentially, text.

Each I/O expression describes how the writing of one parameter e_i will be done using the character %

It can have a variable number of parameters, but

The number of e_i has to correspond to the number of I/O expressions specified in the format

Result

Number of displayed characters (value < 0 in case of an error)

Formated write: printf(...)

Syntax

```
int printf(format,e1,e2,...,en);
```

Description

Writes (displays) the parameters $e_1, e_2, \dots, e_n$ to the standard output based on the parameter format

The format consists of I/O expressions and, potentially, text

Each I/O expression describes how the writing of one parameter e_i will be done using the character %

%

It can have a variable number of parameters, but

The number of e_i has to correspond to the number of I/O expressions specified in the format

Result

Number of displayed characters (value < 0 in case of an error)

#184

1.10 Backup: Formated read: scanf(...)

Original slide 185

Syntax

```
int scanf(format,a1,a2,...,an);
```

Description

Reads from the standard input (keyboard) the values and interprets them based on the parameter format

It assigns each read value to the variables, whose addresses are given by the parameters $a_1, a_2, \dots, a_n$

Address of a variable is noted by the operand & $\&nom_variable$

It can have a variable number of parameters, but

The number of a_i has to correspond to the number of I/O expressions specified in the format

Result

Returns the number of elements correctly assigned

Formatted read: scanf(...)

Syntax

```
int scanf(format a1, a2, ..., an);
```

Description

Reads from the standard input (keyboard) the values and interprets them based on the parameter

format

It assigns each read value to the variables, whose addresses are given by the parameters

a1, a2, ..., an

Address of a variable is noted by the operand & &nom_variable

It can have a variable number of parameters, but

The number of a_i has to correspond to the number of I/O expressions specified in the format

Result

Returns the number of elements correctly assigned

#185

1.11 Backup: Format of I/O expressions

Original slide 186

%d: implies that the I/O is a **signed** integer (**int**)

%c : implies that the I/O is an ASCII character (**char**)

%f : implies that the I/O is a **float** (**float**).

%lf : implies that the I/O is a **double** (**double**).

%x: implies that the I/O is a **unsigned** integer to be printed using the hexadecimal representation

%s : implies that the I/O is a string

....

More info: man format or google printf

Remarks

The format can also have normal text (whatever does not have the % character)

Format of I/O expressions

`%d`: implies that the I/O is a signed integer (int)
`%c`: implies that the I/O is an ASCII character (char)
`%f`: implies that the I/O is a float (float).
`%lf`: implies that the I/O is a double (double).
`%x`: implies that the I/O is a unsigned integer to be printed using the hexadecimal representation
`%s`: implies that the I/O is a string

....
More info: man format or google printf

Remarks

The format can also have normal text (whatever does not have the % character)

#186

1.12 Backup: Example

Original slide 187

Example

```
int main()
{
    int i;
    float f;

    printf("Entrez un entier :");
    scanf("%d",&i);

    printf("Entrez un nombre réel :");
    scanf("%f",&f);

    printf("Vous avez saisi %d et %f \n", i, f);
}
```

#187

1.13 Backup: What happens if we use a wrong format?

Original slide 188

- Usually we have only a compiler warning
- Question : What prints the following program?
- Answer: Segmentation fault (core dumped) or unexpected behaviour
- Tries to print a string of characters stored in the memory starting at the address 1654 and its end is indicated by the `\0` character.

- At the end, the program will try to access an illegal address
- Very common error!

What happens if we use a wrong format?

- Usually we have only a compiler warning
- Question: What prints the following program?
- Answer: Segmentation fault (core dumped) or unexpected behaviour
- Tries to print a string of characters stored in the memory starting at the address 1654 and its end is indicated by the '\0' character.
- At the end, the program will try to access an illegal address
- Very common error!

```
#include <stdio.h>
void main()
{
    int a = 1654 ;
    printf("%s\n",a);
}
```

#188

1.14 Backup: Type conversion: Inbuilt typecast functions

Original slide 189

- atof() : convert string to float
- atoi() : convert string to integer
- atol() : convert string to long
- itoa() : convert integer to string
- ltoa() : convert long to string

Type conversion: Inbuilt typecast functions

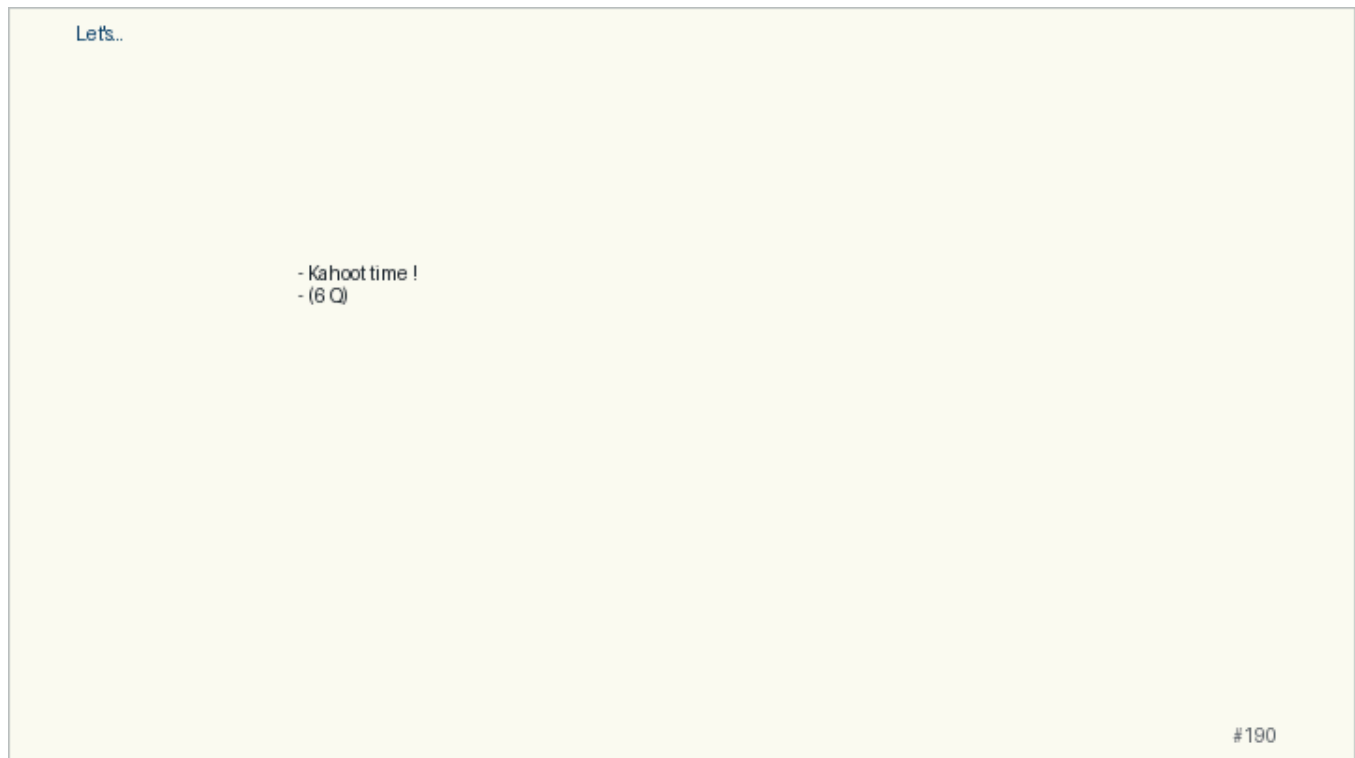
- atof() : convert string to float
- atoi() : convert string to integer
- atol() : convert string to long
- itoa() : convert integer to string
- ltoa() : convert long to string

#189

1.15 Backup: Let's...

Original slide 190

- Kahoot time !
- (6 Q)



2 Control Structures

Live pacing: about 8 min

Question. How does a C program communicate and choose a path?

Core claim. Library calls and control structures turn values into observable behavior.

Target capability. Students can select the right format specifier, branch, and loop for a small task.

2.1 Main Path

- Start from the question: How does a C program communicate and choose a path?
- Build the model: Library calls and control structures turn values into observable behavior.
- End with a checkable action: Students can select the right format specifier, branch, and loop for a small task.
- Keep only this slide on the horizontal path during the live lecture.

- Basic components of C language: Control structures

#191

2.2 Worked Example

- Use this as the live trace: Read one value, validate it, print a formatted result, then add a loop for repeated input.
- Representative source slide: 193
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Conditions: If

```
Syntax: if (expression) {  
list of instructions1  
}
```

#193

2.3 Anecdote Or Motivation

`printf` is a good warning: the compiler trusts the format string more than the value.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Other types of objects

- Derived data types:
 - Pointer type
 - Array type
 - Structure type
 - Union type
- Enumerated types:
 - Arithmetic types used to define variables
 - Variables can only assign certain discrete integer values.
- Function type
- Void type

#198

2.5 Backup Index

Original slides 191-198

- Basic components of C language: Control structures

#191

slide 191

Control structures

- Similar to the ones in JAVA and Scripts
- NO BOOLEAN type!
- Example

```
int i=7;  
while(i) {  
  i-;  
  ...  
}  
int i=7;  
while(i!=0) {  
  i-;  
  ...  
}
```

- Both ways are equivalent, BUT the second one is more readable!

#192

slide 192

Conditions: If

```
Syntax: if (expression) {  
  list of instructions1  
}
```

#193

slide 193

Conditions: ternary instruction

- Syntax:
- expression1 ? expression2 : expression3;
- The value of a ternary instruction (like a multiplexer?) is determined as following:

#194

slide 194

Conditions: switch - case

- Syntax: switch(expression) {
- case constant-expression1:
- list of instructions1;

#195

slide 195

Loops

```
Syntax while while (expression) {  
list of instructions;  
}  
Syntax do-while do {  
list of instructions  
} while (expression);  
Syntax for for(expression1; expression2; expression3) {  
list of instructions;  
}  
expression1: initialization  
expression2: condition of termination  
expression3: step
```

#196

slide 196

Anchors: Basic components of C language: Control structures; Control structures; Conditions: If; Conditions: ternary instruction; Conditions: switch - case; Loops

2.6 Backup: Basic components of C language: Control structures

Original slide 191

- Basic components of C language: Control structures

#191

2.7 Backup: Control structures

Original slide 192

- Similar to the ones in JAVA and Scripts
- NO BOOLEAN type!

- Example

```
int i=7;
while(i) {
  i--;
  ...
}
int i=7;
while(i!=0) {
  i--;
  ...
}
```

- Both ways are equivalent, BUT the second one is more readable!

Control structures

- Similar to the ones in JAVA and Scripts
- NO BOOLEAN type!
- Example

```
int i=7;
while(i) {
  i--;
  ...
}
int i=7;
while(i!=0) {
  i--;
  ...
}
```

- Both ways are equivalent, BUT the second one is more readable!

#192

2.8 Backup: Conditions: If

Original slide 193

```
Syntax: if (expression) {
  list of instructions1
}
if (expression) {
  list of instructions1
} else{
  list of instructions2
}
```

Conditions: If

```
Syntax: if (expression) {  
list of instructions1  
}
```

#193

2.9 Backup: Conditions: ternary instruction

Original slide 194

- Syntax:
- `expression1 ? expression2 : expression3;`
- The value of a ternary instruction (like a multiplexer?) is determined as following:
- `expression1` is evaluated.
- If it is true, then `expression2` is evaluated and becomes the result of the entire expression.
- If it is false, then `expression3` is evaluated and its value becomes the result of the expression.

Conditions: ternary instruction

```
- Syntax:  
- expression1 ? expression2 : expression3;  
- The value of a ternary instruction (like a multiplexer?) is determined as following:
```

#194

2.10 Backup: Conditions: switch - case

Original slide 195

- Syntax: switch(expression) {
- case constant-expression1:
- list of instructions1 ;
- break ; (optional)
- case constant-expression2: list of instruction2 ;
- break ;
- default: (optional)
- list of instructions3 ;
- }

Conditions: switch - case

- Syntax: switch(expression) {
- case constant-expression1:
- list of instructions1 ;

#195

2.11 Backup: Loops

Original slide 196

```
Syntax while while (expression) {  
list of instructions ;  
}  
Syntax do-while do {  
list of instructions  
} while (expression) ;  
Syntax for for(expression1; expression2; expression3) {  
list of instructions ;  
}  
expression1: initialization  
expression2: condition of termination  
expression3: step
```

Loops

```
Syntax while while (expression) {  
list of instructions;  
}  
Syntax do-while do {  
list of instructions  
} while (expression);  
Syntax for for(expression1; expression2; expression3) {  
list of instructions;  
}  
expression1: initialization  
expression2: condition of termination  
expression3: step
```

#196

2.12 Backup: Loops: Control statements

Original slide 197

- Change execution from its normal sequence
- Instruction break;
- Terminates the loop or switch statement
- Transfers execution to the statement immediately following the END of the loop or switch.
- Instruction continue;
- Causes the loop to skip the remaining of its BODY
- Immediately retests the condition expression before reiterating.
- Instruction goto label;
- Recommendation: NEVER USE goto ! (except special cases)

Loops: Control statements

- Change execution from its normal sequence
- Instruction break;
- Terminates the loop or switch statement
- Transfers execution to the statement immediately following the END of the loop or switch.
- Instruction continue;
- Causes the loop to skip the remaining of its BODY
- Immediately retests the condition expression before reiterating.
- Instruction goto label;
- Recommendation: NEVER USE goto ! (except special cases)

#197

2.13 Backup: Other types of objects

Original slide 198

- Derived data types:
- Pointer type
- Array type
- Structure type
- Union type
- Enumerated types:
- Arithmetic types used to define variables
- Variables can only assign certain discrete integer values.
- Function type
- Void type

Other types of objects

- Derived data types
- Pointer type
- Array type
- Structure type
- Union type
- Enumerated types
- Arithmetic types used to define variables
- Variables can only assign certain discrete integer values.
- Function type
- Void type

#198